DAC-MACS: Effective Data Access Control for Multi-Authority Cloud Storage Systems

Kan Yang*†, Xiaohua Jia*, Kui Ren†, Bo Zhang*
*Dept. of CS, City University of Hong Kong
†Dept. of CSE, University at Buffalo, The State University of New York
{kanyang3, Bo.Zhang}@student.cityu.edu.hk, jia@cs.cityu.edu.hk, kuiren@buffalo.edu

***Abstract*—Data access control is an effective way to ensure the data security in the cloud. However, due to data outsourcing and untrusted cloud servers, the data access control becomes a challenging issue in cloud storage systems. Existing access control schemes are no longer applicable to cloud storage systems, because they either produce multiple encrypted copies of the same data or require a fully trusted cloud server. Ciphertext-Policy Attribute-based Encryption (CP-ABE) is a promising technique for access control of encrypted data. It requires a trusted authority manages all the attributes and distributes keys in the system. In cloud storage systems, there are multiple authorities co-exist and each authority is able to issue attributes independently. However, existing CP-ABE schemes cannot be directly applied to data access control for multi-authority cloud storage systems, due to the inefficiency of decryption and revocation. In this paper, we propose DAC-MACS (Data Access Control for Multi-Authority Cloud Storage), an effective and secure data access control scheme with efficient decryption and revocation. Specifically, we construct a new multi-authority CP-ABE scheme with efficient decryption and also design an efficient attribute revocation method that can achieve both forward security and backward security. The analysis and the simulation results show that our DAC-MACS is highly efficient and provably secure under the security model.**

***Index Terms*—Access Control, CP-ABE, Decryption Outsourcing, Attribute Revocation, Multi-authority Cloud.**

I. Introduction

Cloud storage is an important service of cloud computing [1]. It allows data owners to host their data in the cloud and rely on cloud servers to provide "24/7/365" data access to the users (data consumers). Data access control is an effective way to ensure the data security in the cloud. However, cloud storage service separates the roles of the data owner from the data service provider, and the data owner does not interact with the user directly for providing data access service, which makes the data access control a challenging issue in cloud storage systems. Because the cloud server cannot be fully trusted by data owners, existing server-based access control methods are no longer applicable to cloud storage systems. To prevent the untrusted servers from accessing sensitive data, traditional methods usually encrypt the data and only users holding valid keys can access the data. These methods require complicated key management schemes and the data owners have to stay online all the time to deliver keys to new users in the system. Moreover, these methods incur high storage overhead on the server, because the server should store multiple encrypted copies of the same data for users with different keys.

Ciphertext-Policy Attribute-based Encryption (CP-ABE) [2], [3] is regarded as one of the most suitable technologies for data access control in cloud storage systems, because it gives the data owner more direct control on access policies and does not require the data owner to distribute keys. In CP-ABE scheme, there is an authority that is responsible for attribute management and key distribution. The authority can be the registration office in a university, the human resource department in a company, etc. The data owner defines the access policies and encrypts data under the policies. Each user will be issued a secret key reflecting its attributes. A user can decrypt the ciphertexts only when its attributes satisfy the access policies.

Extensive research has been done for single authority systems [2]–[6]. However, in cloud storage systems, a user may hold attributes issued by multiple authorities and the owner may share data with the users administrated to different authorities. For instance, in an E-healthy system, the medical data may be shared only with a user who has the attribute of "Doctor" issued by a hospital and the attribute "Medical Researcher" issued by a medical research center. Although some multi-authority CP-ABE schemes [7]–[10] have been proposed for data encryption, they cannot be directly applied to data access control for multi-authority cloud storage systems, because they either require a global central attribute authority to manage all the attributes across different organizations or lack of efficiency. The aim of this paper is to study the data access control issue in multi-authority cloud storage systems.

One critical requirement in the design of access control schemes is the *efficiency in computation*. There are two operations in access control that require efficient computation, namely *decryption* and *revocation*. The users may use their smart phones to access the data in nowadays cloud storage systems, but the computation ability of smart phones is not as strong as the PCs. Thus, the decryption on each user should be as efficient as possible in data access control schemes. When a user is degraded or leaving the system, some attributes should be revoked from this user. There are two requirements of the efficient attribute revocation: 1) The revoked user (whose attribute is revoked) cannot decrypt the new ciphertexts that require the revoked attributes to decrypt (Backward Security); 2) The newly joined user can also decrypt the previously published ciphertexts that are encrypted with previous public keys if it has sufficient attributes (Forward Security).

978-1-4673-5946-7/13/$31.00 ©2013 IEEE

In this paper, we first construct a new multi-authority CP-ABE scheme with efficient decryption and propose an efficient attribute revocation method for it. Then, we apply them to design an effective access control scheme for multi-authority cloud storage systems. The main contributions of this work can be summarized as follows.

1) We propose DAC-MACS (Data Access Control for Multi-Authority Cloud Storage), an effective and secure data access control scheme for multi-authority cloud storage systems, which is provably secure in the random oracle model and has better performance than existing schemes.
2) We construct a new multi-authority CP-ABE scheme with efficient decryption. Specifically, we outsource the main computation of the decryption by using a token-based decryption method.
3) We also design an efficient immediate attribute revocation method for multi-authority CP-ABE scheme that achieves both forward security and backward security. It is efficient in the sense that it incurs less communication cost and computation cost of the revocation.

The remaining of this paper is organized as follows. We first give the definition of the system model, framework and security model in Section II. Then, we propose a new multi-authority CP-ABE scheme with efficient decryption and revocation, and then apply it to design DAC-MACS in Section III. In Section IV, we analyze DAC-MACS in terms of both the security and the performance. Section V gives the related work on data access control and the attribute revocation in ABE systems. Finally, the conclusion is given in Section VI.

II. System Model and Security Model

A. Definition of System Model

We consider a cloud storage system with multiple authorities, as shown in Fig.1. The system model consists of five types of entities: a global certificate authority (CA), the attribute authorities (*AA*s), the cloud server (server), the data owners (owners) and the data consumers (users).

The CA is a global trusted certificate authority in the system. It sets up the system and accepts the registration of all the users and *AA*s in the system. For each legal user in the system, the CA assigns a global unique user identity to it and also generates a pair of global secret key and global public key for this user. However, the CA is *not* involved in any attribute management and any generation of secret keys that are associated with attributes. For example, the CA can be the Social Security Administration, an independent agency of the United States government. Each user will be issued a Social Security Number (SSN) as its global identity.

Every *AA* is an independent attribute authority that is responsible for issuing, revoking and updating user's attributes according to their role or identity in its domain. In DAC-MACS, every attribute is associated with a single *AA*, but each *AA* can manage an arbitrary number of attributes. Every *AA* has full control over the structure and semantics of its attributes.

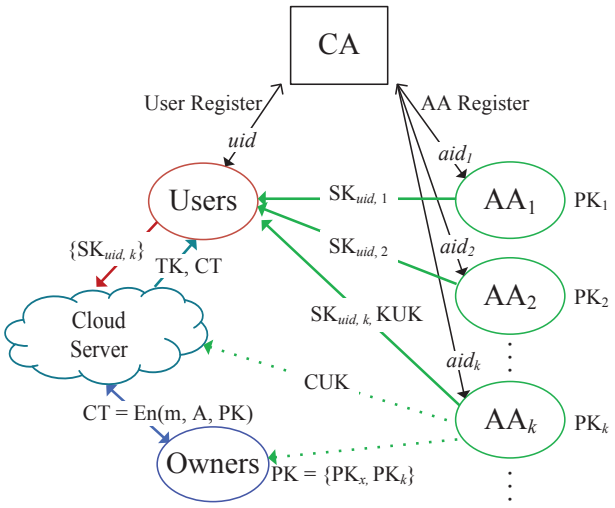


Fig. 1. System Model of DAC-MACS

Each *AA* is responsible for generating a public attribute key for each attribute it manages and a secret key for each user reflecting their attributes.

The cloud server stores the owners' data and provides data access service to users. It helps the user decrypt a ciphertext by generating a decryption token of the ciphertext according to user's secret keys issued by the *AA*s. The server also does the ciphertext update when an attribute revocation happens.

Each owner first divides the data into several components according to the logic granularities and encrypts each data component with different content keys by using symmetric encryption techniques. Then, the owner defines the access policies over attributes from multiple attribute authorities and encrypts the content keys under the policies. Then, the owner sends the encrypted data to the cloud server together with the ciphertexts[1]. They do not rely on the server to do data access control. Instead, the ciphertext can be accessed by all the legal users in the system, which means that any legal user who has been authenticated by the system somehow, he/she can freely query any interested ciphertexts from the server. But, the *access control happens inside the cryptography*. That is only when the user's attributes satisfy the access policy defined in the ciphertext, the user is able to decrypt the ciphertext. Thus, users with different attributes can decrypt different number of content keys and thus obtain different granularities of information from the same data.

Each user is assigned with a global user identity from the CA and can freely get the ciphertexts from the server. To decrypt a ciphertext, each user may submit their secret keys issued by some *AA*s together with its global public key to the server and ask it to generate a decryption token for each ciphertext. Upon receiving the decryption token, the user can use it to decrypt the ciphertext together with its global secret key. Only when the user's attributes satisfy the access policy

[1]In this paper, we simply use the ciphertext to denote the encrypted content keys with CP-ABE.

defined in the ciphertext, the server can generate the correct decryption token. The secret keys and the global user's public key can be stored on the server; subsequently, the user does not need to submit any secret keys if no secret keys are updated for the further decryption token generation.

B. Definition of DAC-MACS Framework

The framework of DAC-MACS is defined as follows.

Definition 1 (DAC-MACS)**.** *DAC-MACS is a collection of algorithms that combines a set of CP-ABE algorithms:* CASetup, AASetup, SKeyGen, Encrypt, TKGen, Decrypt *and a set of attribute revocation algorithms:* UKeyGen, SKUpdate *and* CTUpdate.

- **CASetup**$(1^\lambda) \rightarrow (\mathsf{MSK}, \mathsf{SP}, sk_{CA}, vk_{CA}, \{(uid, \mathsf{GPK}_{uid}, \mathsf{GSK}_{uid}, Cert(uid))\})$. The CA setup algorithm is run by the CA. It takes no input other than the implicit security parameter λ. It outputs the master key MSK, the system parameter SP, a secret and verificative key pair (sk_{CA}, vk_{CA}) of CA. For each user *uid*, it generates a global public/secret key pair $(\mathsf{GPK}_{uid}, \mathsf{GSK}_{uid})$ and a certificate $Cert(uid)$.
- **AASetup**$(aid) \rightarrow (\mathsf{SK}_{aid}, \mathsf{PK}_{aid}, \{\mathsf{VK}_{x_{aid}}, \mathsf{PK}_{x_{aid}}\})$. The attribute authority setup algorithm is run by each *AA*. It takes the authority identity *aid* as input. It outputs a pair of authority secret key SK_{aid} and authority public key PK_{aid}, the set of version keys and public attribute keys $\{\mathsf{VK}_{x_{aid}}, \mathsf{PK}_{x_{aid}}\}$ for all attributes x issued by the AA_{aid}.
- **SKeyGen**$(S_{uid,aid}, \mathsf{SK}_{aid}, \{\mathsf{PK}_{x_{aid}}\}, \mathsf{SP}, Cert(uid)) \rightarrow \mathsf{SK}_{uid,aid}$. The secret key generation algorithm is run by each *AA*. It takes as inputs a set of attributes $S_{uid,aid}$ that describes the secret key, the authority secret key SK_{aid}, the set of public attribute keys $\{\mathsf{PK}_{x_{aid}}\}$, the system parameter SP and the certificate of the user with *uid*. It outputs a secret key $\mathsf{SK}_{uid,aid}$ for the user with *uid*.
- **Encrypt**$(\mathsf{SP}, \{\mathsf{PK}_k\}_{k\in I_A}, \{\mathsf{PK}_{x_k}\}_{x_k\in S_{A_k}}^{k\in I_A}, m, \mathbb{A}) \rightarrow CT$. The encryption algorithm is run by data owners. It takes as inputs the system parameter SP, a set of public keys $\{\mathsf{PK}_k\}_{k\in I_A}$ from the involved authority set I_A, a set of public attribute keys $\{\mathsf{PK}_{x_k}\}_{x_k\in S_{A_k}}^{k\in I_A}$, a message m and an access structure $\mathbb{A}$ over all the selected attributes from the involved *AA*s. The algorithm encrypts m according to the access structure $\mathbb{A}$ and outputs a ciphertext CT. We will assume that the ciphertext implicitly contains the access structure $\mathbb{A}$.
- **TKGen**$(\mathsf{CT}, \mathsf{GPK}_{uid}, \{\mathsf{SK}_{uid,k}\}_{k\in I_A}) \rightarrow \mathsf{TK}$. The decryption token generation algorithm is run by the cloud server. It takes as inputs the ciphertext CT which contains an access structure $\mathbb{A}$, user's global public key GPK_{uid} and a set of user's secret keys $\{SK_{uid,k}\}_{k\in I_A}$. If the set of attributes S satisfies the access structure $\mathbb{A}$, the algorithm can successfully compute the correct decryption token TK of the ciphertext.
- **Decrypt**$(\mathsf{CT}, \mathsf{TK}, \mathsf{GSK}_{uid}) \rightarrow m$. The decryption algorithm is run by the users. It takes as inputs the ciphertext CT, the decryption token TK and the user's global secret key GSK_{uid}. It outputs the message m.
- **UKeyGen**$(\mathsf{SK}_{aid}, \{u_j\}, \mathsf{VK}_{\tilde{x}_{aid}}) \rightarrow (\mathsf{KUK}_{j,\tilde{x}_{aid}}, \mathsf{CUK}_{\tilde{x}_{aid}})$. The update key generation algorithm is run by the *AA* corresponding to the revoked attribute $\tilde{x}_{aid}$. It takes as inputs the authority secret key SK_{aid}, a set of user's secret $\{u_j\}$ and the previous version key of the revoked attribute $\mathsf{VK}_{\tilde{x}_{aid}}$. It outputs both the user's Key Update Key $\mathsf{KUK}_{j,\tilde{x}_{aid}}(j \in S_U, j \neq \mu, \tilde{x}_{aid} \in S_{j,aid})$ and the Ciphertext Update Key $\mathsf{CUK}_{\tilde{x}_{aid}}$.
- **SKUpdate**$(\mathsf{SK}_{uid,aid}, \mathsf{KUK}_{uid,\tilde{x}_{aid}}) \rightarrow \mathsf{SK}'_{uid,aid}$. The user's secret key update algorithm is run by all the non-revoked users. It takes as inputs the current secret key $\mathsf{SK}_{uid,aid}$ and its key update key $\mathsf{KUK}_{uid,\tilde{x}_{aid}}$. It outputs a new secret key $\mathsf{SK}'_{uid,aid}$.
- **CTUpdate**$(\mathsf{CT}, \mathsf{CUK}_{\tilde{x}_{aid}}) \rightarrow \mathsf{CT}'$. The ciphertext update algorithm is run by the cloud server. It takes as inputs the current ciphertext CT and the ciphertext update key $\mathsf{CUK}_{\tilde{x}_{aid}}$. It outputs a new ciphertext CT′.

C. Definition of Security Model

In multi-authority cloud storage systems, we assume: The CA is trusted in the system, but we still need to prevent it from decrypting any ciphertexts. Each *AA* is also trusted, but it can be corrupted by the adversary. The server is curious but honest. It is curious about the content of the encrypted data or the received message, but will execute correctly the task assigned by each *AA*. The users are dishonest and may collude to obtain unauthorized access to data.

We now describe the security model of DAC-MACS by the following game between a challenger and an adversary. The security model allows the adversary to query for any secret keys and update keys that cannot be used to decrypt the challenge ciphertext. Similar to [10], we assume that the adversaries can corrupt authorities only statically, but key queries are made adaptively. Let S_A denote the set of all the authorities. The security game is defined as follows.

Setup. The system parameters are generated by running the CA setup algorithm. The adversary specifies a set of corrupted attribute authorities $S'_A \subset S_A$. The challenger generates the public keys by querying the *AA* setup oracle, and generates the secret keys by querying the secret key generation oracle. For uncorrupted authorities in $S_A - S'_A$, the challenger sends only the public keys to the adversary. For corrupted authorities in S'_A, the challenger sends both public keys and secret keys to the adversary.

Phase 1. The adversary makes secret key queries by submitting pairs (uid, S_{uid}) to the challenger, where $S_{uid} = \{S_{uid,k}\}_{k\in S_A - S'_A}$ is a set of attributes belonging to several uncorrupted *AA*s. The challenger gives the corresponding secret keys $\{\mathsf{SK}_{uid,k}\}$ to the adversary. The adversary also makes update key queries by submitting a set of attributes S'_{aid}. The challenger responses the corresponding update keys to the adversary.

Challenge. The adversary submits two equal length messages m_0 and m_1. In addition, the adversary gives a challenge

access structure (M^*, ρ^*) which must satisfy the following constraints. We let V denote the subset of rows of M^* labeled by attributes controlled by corrupted *AA*s. For each *uid*, we let V_{uid} denote the subset of rows of M^* labeled by attributes that the adversary has queried. For each *uid*, we require that the subspace spanned by $V \cup V_{uid}$ must not include $(1,0,\ldots,0)$. In other words, the adversary cannot ask for a set of keys that allow decryption, in combination with any keys that can obtained from corrupted *AA*s. The challenger then flips a random coin b, and encrypts m_b under the access structure (M^*, ρ^*). Then, the ciphertext CT^* is given to the adversary.

Phase 2. The adversary may query more secret keys and update keys, as long as they do not violate the constraints on the challenge access structure (M^*, ρ^*) and the following constraints: None of the updated secret keys (generated by the queried update keys and the queried secret keys[2]) are able to decrypt the challenged ciphertexts. In other words, the adversary is not able to query the update keys that can update the queried secret keys to the new secret keys that can decrypt the challenge ciphertext.

Guess. The adversary outputs a guess b' of b.

The advantage of an adversary $\mathcal{A}$ in this game is defined as $Pr[b' = b] - \frac{1}{2}$.

Definition 2. *DAC-MACS is secure against static corruption of authorities if all polynomial time adversaries have at most a negligible advantage in the above security game.*

III. DAC-MACS: Data Access Control for Multi-Authority Cloud Storage

In this section, we first give an overview of the challenges and techniques of designing access control schemes for multi-authority cloud storage systems. Then, we propose the detailed construction of our access control scheme DAC-MACS with efficient decryption and revocation.

A. Overview of Our Solutions

Although the existing multi-authority CP-ABE scheme [10] proposed by Lewko and Waters has high policy expressiveness and has been extended to support attribute revocation in [11], it still cannot be applied to access control for multi-authority cloud storage systems due to the inefficiency of decryption and revocation. Thus, the main challenge is to design a new underlying multi-authority CP-ABE scheme with efficient decryption and revocation.

One challenging issue in the design of a multi-authority CP-ABE scheme is how to tie the secret keys together and prevent the collusion attack. Similar to [7], in DAC-MACS, we separate the authority into a global certificate authority (CA) and multiple attribute authorities (*AA*s). The CA sets up the system and assigns a global user identity *uid* to each user and a global authority identity *aid* to each attribute authority in the system. Since the *uid* is global unique in the system, secret keys issued by different *AA*s for the same *uid* can be tied together for decryption. Also, since each *AA* is associated with an *aid*, every attribute is distinguishable even though some *AA*s may issue the same attribute. Thus, the collusion attack can be resisted by using the *aid* and *uid*. However, different from [7], the CA in DAC-MACS is *not* involved in any attribute management and the creation of secret keys reflecting the user's attributes. Our scheme also requires all the *AA*s to generate their own public keys and uses them to encrypt data together with the global public parameters, instead of using the system unique public key (generated by the unique master key) to encrypt data. This solves the security drawback in [7], i.e., it prevents the CA from decrypting the ciphertexts.

To achieve efficient decryption on the user, we propose a token-based decryption outsourcing method. We apply the decryption outsourcing idea from [11] and extend it to multiple authority systems by letting the CA generate a global secret/public key pair for each legal user in the system. During the decryption, the user submits its secret keys issued by *AA*s to the server and asks the server to compute a decryption token for the ciphertext. The user can decrypt the ciphertext by using the decryption token together with its global secret key.

To solve the attribute revocation problem, we assign a version number for each attribute, such that when an attribute revocation happens, only those components associated with the revoked attribute in secret keys and ciphertexts need to be updated. When an attribute of a user is revoked from any *AA*, the *AA* generates a new version key for this revoked attribute and generate the update keys which contains a ciphertext update key and a set of user's key update keys. With the user's key update key, each non-revoked user who holds the revoked attributes can update its secret key. Because the update keys are associated with the *uid*, the revoked user cannot update its secret key by using other users' update keys (Backward Security). By using the ciphertext update key, the components associated with the revoked in the ciphertext can be updated to the current version. To improve the efficiency, we delegate the workload of ciphertext update to the server by using the proxy re-encryption method, such that the newly joined user is also able to decrypt the previous data which are published before it joins the system (Forward Security). Moreover, all the users need to hold only the latest secret key, rather than to keep records on all the previous secret keys.

B. Construction of DAC-MACS

Let S_A and S_U denote the set of attribute authorities and the set of users in the system respectively. Let $\mathbb{G}$ and $\mathbb{G}_T$ be the multiplicative groups with the same prime order p and $e : \mathbb{G} \times \mathbb{G} \to \mathbb{G}_T$ be the bilinear map. Let g be the generator of $\mathbb{G}$. Let $H : \{0,1\}^* \to \mathbb{G}$ be a hash function such that the security is in the random oracle.

The DAC-MACS consists of five phases: System Initialization, Secret Key Generation by *AA*s, Dat Encryption by Owners, Data Decryption by Users (with the help of Cloud Server) and Attribute Revocation. For clarity, the attribute revocation phase will be described in the next section.

[2]There is another reason that makes the queried secret keys cannot decrypt the challenge ciphertext. That is at least one of the attributes in the previous queried secret keys may be not in the current version.

Phase 1: System Initialization

There are two steps in the system initialization phase: CA Setup and *AA* Setup.

1) *CA Setup*

The CA runs the CA setup algorithm $\mathsf{CASetup}$, which takes a security parameter as input. The CA first chooses a random number $a \in \mathbb{Z}_p$ as the master key MSK of the system and compute the system parameter $\mathsf{SP} = g^a$. Then, the CA generates a pair of secret key and verificative key (sk_{CA}, vk_{CA}). The CA accepts both *User Registration* and *AA Registration*.

- *User Registration*
  Every user should register itself to the CA during the system initialization. If the user is legal in the system, the CA then assigns a global unique user identity *uid* to this user. For each user with *uid*, it generates the global public key $\mathsf{GPK}_{uid} = g^{u_{uid}}$ and the global secret key $\mathsf{GSK}_{uid} = z_{uid}$ by randomly choosing two numbers $u_{uid}, z_{uid} \in \mathbb{Z}_p$. The CA also generates a certificate $Cert(uid)$ which contains an item $En_{sk_{CA}}(uid, u_{uid}, g^{1/z_{uid}})$. Then, the CA gives the global public key GPK_{uid}, the global secret key GSK_{uid} and the user's certificate $Cert(uid)$ to this user.
- *AA Registration*
  Each *AA* should also register itself to the CA during the system initialization. If the *AA* is a legal authority in the system, the CA first assigns a global authority identity *aid* to this *AA*. Then, the CA sends both its verificative key vk_{CA} and the system parameter SP to this *AA*.

2) *AA Setup*

Each $AA_k (k \in S_A)$ runs the *AA* setup algorithm $\mathsf{AASetup}$. Let S_{A_k} denote the set of all attributes managed by this authority AA_k. It chooses three random numbers $\alpha_k, \beta_k, \gamma_k \in \mathbb{Z}_p$ as the authority secret key $\mathsf{SK}_k = (\alpha_k, \beta_k, \gamma_k)$. For each attribute $x_k \in S_{A_k}$, the authority generates a public attribute key as $\mathsf{PK}_{x_k} = (g^{v_{x_k}} H(x_k))^{\gamma_k}$ by implicitly choosing an attribute version key as $\mathsf{VK}_{x_k} = v_{x_k}$. The AA_k also computes the authority public key as

$$\mathsf{PK}_k = \left(e(g,g)^{\alpha_k},\ g^{\frac{1}{\beta_k}},\ g^{\frac{\gamma_k}{\beta_k}} \right).$$

All the public attribute keys are published on the public bulletin board of AA_k, together with the authority public key PK_k.

Phase 2: Secret Key Generation by *AA*s

For every user $U_j (j \in S_U)$, each $AA_k (k \in S_A)$ first authenticates whether this user is a legal user by verifying the certificate of the user. It decrypts the item $En_{sk_{CA}}(uid_j, u_j, g^{1/z_j})$ in the certificate $Cert(uid_j)$ by using vk_{CA} and authenticates the user. If the user is not a legal user, it aborts. Otherwise, the AA_k assigns a set of attributes $S_{j,k}$ to this user according to its role or identity in its administration domain. Then, the AA_k runs the secret key generation algorithm $\mathsf{SKeyGen}$ to generate the user's secret key $SK_{j,k}$ as

$$\mathsf{SK}_{j,k} = (\ K_{j,k} = g^{\frac{\alpha_k}{z_j}} \cdot g^{a u_j} \cdot g^{\frac{a}{\beta_k} t_{j,k}},\ L_{j,k} = g^{\frac{\beta_k}{z_j} t_{j,k}},\ R_{j,k} = g^{a t_{j,k}},$$
$$\forall x_k \in S_{j,k} : K_{j,x_k} = g^{\frac{\beta_k \gamma_k}{z_j} t_{j,k}} \cdot (g^{v_{x_k}} \cdot H(x_k))^{\gamma_k \beta_k u_j}\ ).$$

where $j \in S_U$, $k \in S_A$, and $t_{j,k}$ is a random number in $\mathbb{Z}_p$.

Phase 3: Data Encryption by Owners

The owner first encrypts the data component with a content key by using symmetric encryption methods. It then runs the encryption algorithm $\mathsf{Encrypt}$ to encrypt the content key. It takes as inputs the system parameter, the public keys, the content key κ and an access structure (M, ρ) over all the selected attributes from the involved *AA*s. Let M be a $l \times n$ matrix, where l denotes the total number of all the attributes. The function ρ associates rows of M to attributes.

The encryption algorithm first chooses a random encryption exponent $s \in \mathbb{Z}_p$ and chooses a random vector $\vec{v} = (s, y_2, \cdots, y_n) \in \mathbb{Z}_p^n$, where $y_2, \cdots, y_n$ are used to share the encryption exponent s. For $i = 1$ to l, it computes $\lambda_i = \vec{v} \cdot M_i$, where M_i is the vector corresponding to the i-th row of M. Then, it randomly chooses $r_1, r_2, \cdots, r_l \in \mathbb{Z}_p$ and computes the ciphertext as

$$\mathsf{CT} = (\ C = \kappa \cdot (\prod_{k \in I_A} e(g,g)^{\alpha_k})^s,\ C' = g^s,\ C'' = g^{\frac{s}{\beta_k}},$$
$$\forall i = 1\ to\ l : C_i = g^{a\lambda_i} \cdot ((g^{v_{\rho(i)}} H(\rho(i)))^{\gamma_k})^{-r_i},$$
$$D_{1,i} = g^{\frac{r_i}{\beta_k}},\ D_{2,i} = g^{-\frac{\gamma_k}{\beta_k} r_i},\ \rho(i) \in S_{A_k}\ ).$$

Phase 4: Data Decryption by Users (with the help of Cloud Server)

All the legal users in the system can query any interested encrypted data from the cloud server. But only when the user's attributes satisfy the access structure embedded in the ciphertext, he/she can decrypt the content key and use it to further decrypt the data component. The decryption phase consists of two steps:

Step 1. Token Generation by Cloud Server

The user $U_j (j \in S_U)$ sends its secret keys $\{\mathsf{SK}_{j,k}\}_{k \in S_A}$ to the server and asks the server to compute a decryption token for the ciphertext CT by running the token generation algorithm TKGen. Only when the attributes the user U_j possesses satisfy the access structure defined in the ciphertext CT, the server can successfully compute the correct decryption token TK.

Let $I = \{I_{A_k}\}_{k \in I_A}$ be the whole index set of all the attributes involved in the ciphertext, where $I_{A_k} \subset \{1, \cdots, l\}$ is the index subset of the attributes from the AA_k, defined as $I_{A_k} = \{i : \rho(i) \in S_{A_k}\}$. Let $N_A = |I_A|$ be the number of *AA*s involved in the ciphertext. It chooses a set of constants $\{w_i \in \mathbb{Z}_p\}_{i \in I}$ and reconstructs the encryption exponent as $s = \sum_{i \in I} w_i \lambda_i$ if $\{\lambda_i\}$ are valid shares of the secret s according to M.

The algorithm computes the decryption token TK as

$$\mathsf{TK} = \prod_{k \in I_A} \frac{e(C', K_{j,k}) \cdot e(R_{j,k}, C'')^{-1}}{\prod_{i \in I_{A_k}} \left(e(C_i, \mathsf{GPK}_{U_j}) \cdot e(D_{1,i}, K_{j,\rho(i)}) \cdot e(D_{2,i}, L_{j,k}) \right)^{w_i N_A}}$$
$$= \frac{e(g,g)^{a u_j s N_A} \cdot \prod_{k \in I_A} e(g,g)^{\frac{\alpha_k}{z_j} s}}{e(g,g)^{u_j a N_A \sum_{i \in I} \lambda_i w_i}}$$
$$= \prod_{k \in I_A} e(g,g)^{\frac{\alpha_k}{z_j} s}.$$

It outputs the decryption token TK for the ciphertext CT and sends it to the user U_j.

Step 2. Data Decryption by Users

Upon receiving this decryption token TK, the user U_j can use it to decrypt the ciphertext together with its global secret key $\mathsf{GSK}_{U_j} = z_j$ as

$$\kappa = C / \mathsf{TK}^{z_j}$$

Then, the user can use the content key κ to further decrypt the encrypted data component.

C. Efficient Attribute Revocation for DAC-MACS

Suppose an attribute $\tilde{x}_k$ of the user U_μ is revoked from the AA_k. The attribute revocation includes three phases: *Update Key Generation by AAs*, *Secret Key Update by Non-revoked Users*[3] and *Ciphertext Update by Cloud Server*. The secret key update can prevent the revoked user from decrypting the new ciphertexts which are encrypted by the new public attribute keys (Backward Security). The ciphertext update can make sure that the newly joined user can still access the previous data which is published before it joins the system, when its attributes satisfy the access policy associated with the ciphertext (Forward Security).

Phase 1. Update Key Generation by *AA*s

The corresponding authority AA_k runs the update key generation algorithm $\mathsf{UKeyGen}$ to compute the update keys. The algorithm takes as inputs the authority secret key SK_k, the current attribute version key $v_{\tilde{x}_k}$ and the user's global public keys GPK_{U_j}. It generates a new attribute version key $\mathsf{VK}'_{\tilde{x}_k} = v'_{\tilde{x}_k}$. It first calculates the Attribute Update Key as $\mathsf{AUK}_{\tilde{x}_k} = \gamma_k(v'_{\tilde{x}_k} - v_{\tilde{x}_k})$, then it applies this $\mathsf{AUK}_{\tilde{x}_k}$ to compute the user's Key Update Key $\mathsf{KUK}_{j,\tilde{x}_k} = g^{u_j \beta_k \cdot \mathsf{AUK}_{\tilde{x}_k}}$ and the Ciphertext Update Key as $\mathsf{CUK}_{\tilde{x}_k} = \frac{\beta_k}{\gamma_k} \cdot \mathsf{AUK}_{\tilde{x}_k}$. Then, the AA_k updates the public attribute key of the revoked attribute $\tilde{x}_k$ as $\mathsf{PK}'_{\tilde{x}_k} = \mathsf{PK}_{\tilde{x}_k} \cdot g^{\mathsf{AUK}_{\tilde{x}_k}}$ and broadcasts a message for all the owners that the public attribute key of the revoked attribute $\tilde{x}_k$ is updated. Then, all the owners can get the new public attribute key for the revoked attribute from the public board of AA_k. It outputs both the user's key update key $\mathsf{KUK}_{j,\tilde{x}_k} (j \in S_U, j \neq \mu, \tilde{x}_k \in S_{j,k})$ and the ciphertext update key $\mathsf{CUK}_{\tilde{x}_k}$.

Phase 2. Secret Key Update by Non-revoked Users

For each non-revoked user $U_j (j \in S_U, j \neq \mu)$ who has the attribute $\tilde{x}_k$, the AA_k sends the corresponding user's key update key $\mathsf{KUK}_{j,\tilde{x}_k}$ to it. Upon receiving the user's key update key $\mathsf{KUK}_{j,\tilde{x}_k}$, the user U_j runs the key update algorithm $\mathsf{SKUpdate}$ to update its secret key as

$$\mathsf{SK}'_{j,k} = (\ K'_{j,k} = K_{j,k},\ L'_{j,k} = L_{j,k},\ R'_{j,k} = R_{j,k},\ K'_{j,\tilde{x}_k} = K_{j,\tilde{x}_k} \cdot \mathsf{KUK}_{j,\tilde{x}_k},\ \forall x \in S_u, x \neq \tilde{x} : K'_{j,k} = K_{j,k}\ ).$$

Note that each $\mathsf{KUK}_{j,\tilde{x}_k}$ is associated with the *uid*, so that they are distinguishable for different non-revoked users. Thus, the revoked user U_μ cannot use any other user's update keys to update its secret key.

[3]We use *Non-revoked Users* to denote the set of users who possess the revoked attribute but have not been revoked.

Phase 3. Ciphertext Update by Cloud Server

The AA_k sends a ciphertext update key $\mathsf{CUK}_{\tilde{x}_k}$ to the server. Upon receiving the $\mathsf{CUK}_{\tilde{x}_k}$, the server runs the ciphertext update algorithm $\mathsf{CTUpdate}$ to update all the ciphertexts which are associated with the revoked attribute $\tilde{x}_k$. It takes inputs as the current ciphertext CT and the $\mathsf{CUK}_{\tilde{x}_k}$. It only needs to update only a few components of the ciphertext, which are associated with the revoked attribute $\tilde{x}_k$. The new ciphertext CT' is published as

$$\begin{aligned}\mathsf{CT}' = (\ & C = \kappa \cdot (\prod_{k \in I_A} e(g,g)^{\alpha_k})^s,\ C' = g^s,\ C'' = g^{\frac{s}{\beta_k}},\\ & \forall i = 1\ to\ l:\quad if\ \rho(i) \neq \tilde{x}_k:\ C_i = g^{a\lambda_i} \cdot ((g^{v_{x_k}} H(x_k))^{\gamma_k})^{-r_i},\\ & \qquad D_{1,i} = g^{\frac{r_i}{\beta_k}},\ D_{2,i} = g^{-\frac{\gamma_k}{\beta_k} r_i},\\ & \qquad if\ \rho(i) = \tilde{x}_k:\ C'_i = C_i \cdot D_{2,i}^{\mathsf{CUK}_{\tilde{x}_k}},\\ & \qquad D_{1,i} = g^{\frac{r_i}{\beta_k}},\ D_{2,i} = g^{-\frac{\gamma_k}{\beta_k} r_i}\ ).\end{aligned}$$

DAC-MACS requires to update only a few components which are associated with the revoked attribute, while the other components are not changed. This can greatly improve the efficiency of attribute revocation.

The ciphertext update not only can guarantee the backward security of the attribute revocation, but also can reduce the storage overhead on the users (i.e., all the users need to hold only the latest secret key, rather than to keep records on all the previous secret keys).

IV. Analysis of Our DAC-MACS

This section provides a comprehensive analysis of DAC-MACS, followed by security and performance analysis.

A. Comprehensive Analysis

Let $|p|$ be the size of element in the groups with the prime order p. Let t_c be the total number of attributes in a ciphertext and t_u be the total number of attributes of a user. Let n_u denote the number of users in the system. For the revoked attribute x, let $n_{non,x}$ be the number of non-revoked users who hold the revoked attribute and let $n_{c,x}$ be the number of ciphertexts which contain the revoked attribute.

Table I shows the comparison among our DAC-MACS and two existing schemes, which are all based on the ciphertext re-encryption to achieve the attribute revocation. From the table, we can see that DAC-MACS incurs less computation cost for the decryption on the user and less communication cost for the revocation. In DAC-MACS, the attribute revocation is controlled and enforced by each *AA* independently, but the ciphertexts are updated by the semi-trusted server, which can greatly reduce the workload on the owners. For the security of attribute revocation, DAC-MACS can achieve both forward security and backward security. The cloud server in our system is required to be semi-trusted. Even if the cloud server is not semi-trusted in some scenarios, the server will not update the ciphertexts correctly. In this situation, the forward security cannot be guaranteed, but our system can still achieve the backward security.

TABLE I
COMPREHENSIVE COMPARISON OF CP-ABE WITH ATTRIBUTE REVOCATION SCHEMES

Scheme	Authority	Computation		Revocation	Revocation Security		Revocation	Ciphertext
		Encrypt	Decrypt*	Message ($\|p\|$)	Backward	Forward	Controller	Updater
Hur's [12]	Single	$O(t_c + \log n_u)$	$O(t_u)$	$O(n_{non,x} \log \frac{n_u}{n_{non,x}})$	Yes	Yes	Server†	Server†
DACC [13]	Multiple	$O(t_c)$	$O(t_u)$	$O(n_{c,x} \cdot n_{non,x})$	Yes	No	Owner	Owner
DAC-MACS	Multiple	$O(t_c)$	$O(1)$	$O(n_{non,x})$	Yes	Yes	*AA*	Server‡

$*$: The decryption computation on the user; †: The server is fully trusted; ‡: The server is semi-trusted.

B. Security Analysis

Under the security model we defined in Section II-C, we conclude the security analysis into the following theorems:

Theorem 1. *When the decisional q-parallel BDHE assumption holds, no polynomial time adversary can selectively break our system with a challenge matrix of size $l^* \times n^*$, where $n^* \leq q-1$.*

Proof: Suppose we have an adversary $\mathcal{A}$ with non-negligible advantage $\varepsilon = Adv_{\mathcal{A}}$ in the selective security game against our construction and suppose it chooses a challenge matrix M^* with the dimension at most $q-1$ columns. In the security game, the adversary can query any secret keys and update keys that cannot be used for decryption in combination with any keys it can obtain from the corrupted *AA*s. With these constraints, the security game in multi-authority systems can be treated equally to the one in single authority systems. Similarly, we can build a simulator $\mathcal{B}$ that plays the decisional q-parallel BDHE problem with non-negligible advantage. The detailed proof is described in the full version of this paper [14]. ∎

Theorem 2. *DAC-MACS is secure against the collusion attack.*

Proof: In DAC-MACS, each user in the system is assigned with a global unique identity *uid*, and all the secret keys issued to the same user from different *AA*s are associated with the *uid* of this user. Thus, it is impossible for two or more users to collude and decrypt the ciphertext. Moreover, due to the unique *aid* of each *AA*, all the attributes are distinguishable, even though some *AA*s may issue the same attribute. This can prevent the user from replacing the components of a secret key issued by an *AA* with those components from other secret keys issued by another *AA*. ∎

Privacy-Preserving Guarantee Due to the decryption outsourcing, the server can get the users' secret keys. However, the server still cannot decrypt the ciphertext without the knowledge of the users' global secret keys. Moreover, the ciphertext update is done by using the proxy re-encryption method, thus the server does not need to decrypt the ciphertext.

C. Performance Analysis

We conduct the performance analysis between our DAC-MACS and Ruj's DACC scheme under the metrics of *Storage Overhead*, *Communication Cost* and *Computation Cost*.

TABLE II
COMPARISON OF STORAGE OVERHEAD

Entity	Ruj's DACC [13]	Our DAC-MACS
AA_k	$2n_{a,k}\|p\|$	$(n_{a,k}+3)\|p\|$
Owner	$(n_c + 2\sum_{k=1}^{N_A} n_{a,k})\|p\|$	$(3N_A + 1 + \sum_{k=1}^{N_A} n_{a,k})\|p\|$
User	$(n_{c,x} + \sum_{k=1}^{N_A} n_{a,k,uid})\|p\|$	$(3N_A + 1 + \sum_{k=1}^{N_A} n_{a,k,uid})\|p\|$
Server	$(3t_c+1)\|p\|$	$(3t_c+3)\|p\|$

n_c: total number of ciphertexts stored on the cloud server; $n_{c,x}$:number of ciphertexts contain the revoked attribute x; t_c: total number of attributes in the ciphertext.

1) Storage Overhead

The storage overhead is one of the most significant issues of the access control scheme in cloud storage systems. Suppose there are N_A *AA*s in the system. Let $|p|$ be the element size in the $\mathbb{G}, \mathbb{G}_T, \mathbb{Z}_p$. Let $n_{a,k}$ and $n_{a,k,uid}$ denote the total number of attributes managed by AA_k and the number of attributes assigned to the user with *uid* from AA_k respectively. We compare the storage overhead on each entity in the system, as shown in Table II.

In DAC-MACS, the storage overhead on each AA_k consists of the version number of each attribute and the authority secret key. From Table II, we can see that DAC-MACS incurs less storage overhead on each AA_k than Ruj's DACC scheme, which consists of the secret keys for all the attributes. The public parameters contribute the main storage overhead on the owner. Besides, Ruj's DACC scheme also requires the owner to hold the encryption secret for every ciphertext in the system, because the owner is required to re-encrypt the ciphertexts. This incurs a heavy storage overhead on the owner, especially when the number of ciphertext is large in cloud storage systems. The storage overhead on each user in DAC-MACS comes from the global secret key issued by the CA and the secret keys issued by all the *AA*s. However, in Ruj's DACC scheme, the storage overhead on each user consists of both the secret keys issued by all the *AA*s and the ciphertext components that associated with the revoked attribute, because when the ciphertext is re-encrypted, some of its components related to the revoked attributes should be sent to each non-revoked user who holds the revoked attributes. The ciphertexts contribute the main storage overhead on the server (here we

do not consider the encrypted data which are encrypted by symmetric content keys).

2) Communication Cost

The communication cost of the normal access control is almost the same between our DAC-MACS and Ruj's DACC scheme. Here, we only compare the communication cost of attribute revocation, as shown in Table III. It is easily to find that the communication cost of attribute revocation in Ruj's scheme is linear to the number of ciphertexts which contain the revoked attributes. Due to the large number of ciphertext in cloud storage system, Ruj's scheme incurs a heavy communication cost for attribute revocation.

TABLE III
COMPARISON OF COMMUNICATION COST FOR ATTRIBUTE REVOCATION

Operation	Ruj's DACC [13]	Our DAC-MACS
Key Update	N/A	$n_{non,x}\|p\|$
Ciphertext Update	$(n_{c,x} \cdot n_{non,x} + 1)\|p\|$	$\|p\|$

$n_{non,x}$ is the number of non-revoked users who hold the revoked attribute x; $n_{c,x}$ is the number of ciphertexts which contain the revoked attribute x.

3) Computation Cost

We simulate the computation time of encryption, decryption and ciphertext re-encryption/update in our DAC-MACS and Ruj's DACC scheme. We do the simulation on a Linux system with an Intel Core 2 Duo CPU at 3.16GHz and 4.00GB RAM. The code uses the Pairing-Based Cryptography library version 0.5.12 to simulate the access control schemes. We use a symmetric elliptic curve α-curve, where the base field size is 512-bit and the embedding degree is 2. The α-curve has a 160-bit group order, which means p is a 160-bit length prime. All the simulation results are the mean of 20 trials.

We compare the computation efficiency of both encryption and decryption in two criteria: the number of authorities and the number of attributes per authority, as shown in Fig. 2. Fig.2(a) describes the comparison of encryption time on the owner versus the number of *AA*s, where the involved number of attributes from each *AA* is set to be 10. Fig.2(b) gives the comparison of encryption time on the owner versus the number of attributes from each *AA*, where the involved number of *AA*s is set to be 10. Suppose the user has the same number of attributes from each *AA*. Fig.2(c) shows the comparison of decryption time on the user versus the number of *AA*s, where the number of attributes the user holds from each *AA* is set to be 10. Fig.2(d) describes the comparison of decryption time on the user versus the number of attributes the user holds from each *AA*, where the number of authority for the user is fixed to be 10. Fig.2(e) gives the comparison of ciphertext re-encryption/update versus the number of revoked attributes appeared in the ciphertext.

The simulation results show that our DAC-MACS incurs less computation cost on the encryption of owners, the decryption of users and the re-encryption of ciphertexts.

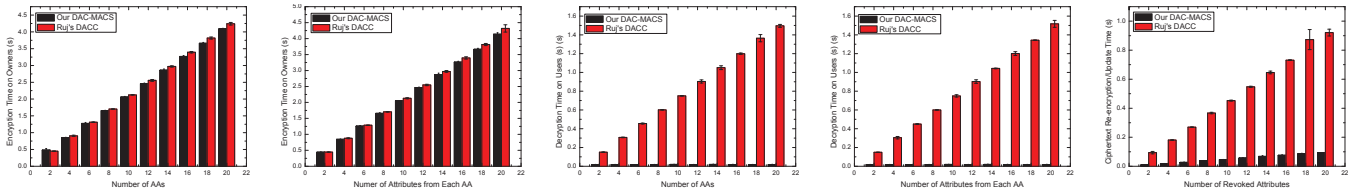
V. RELATED WORK

Cryptographic techniques are well applied to access control for remote storage systems [15]–[17]. To prevent the untrusted servers from accessing sensitive data, traditional methods [18], [19] usually encrypt the data and only the users who hold valid keys can decrypt and access the data. Then, the data access control becomes the matter of key distribution. These methods require complicated key management schemes and the data owners have to stay online all the time to deliver the keys to new user in the system. Moreover, these methods incur high storage overhead on the server, because the server should store multiple encrypted copies of the same data for users with different keys. Some methods [20], [21] deliver the key management and distribution from the data owners to the remote server under the assumption that the server is trusted. However, the server is not fully trusted in cloud storage systems and thus these methods cannot be applied to data access control for cloud storage systems.

Attribute-based Encryption (ABE) is a promising technique that is designed for access control of encrypted data. After Sahai and Waters introduced the first ABE scheme [22], Goyal *et al.* [23] formulated the ABE into two complimentary forms: Key-Policy ABE (KP-ABE) and Ciphertext-Policy ABE (CP-ABE). There are a number of works used ABE to realize fine-grained access control for outsourced data [12], [24], [25]. These schemes require a trusted authority to manage all the attributes in the system and issue secret keys to users. Since the authority can decrypt all the encrypted data, it becomes a vulnerable security point and the performance bottleneck of the system.

Some cryptographic methods are proposed for the multi-authority ABE problem [7]–[10], [26], [27], where there are multiple authorities coexist and the users may have attributes from multiple authorities. However, some of them [7], [8] require a global authority, which would be a vulnerable point for security attacks and a performance bottleneck for large scale systems. In [9], the authors remove the central authority by using a distributed PRF (pseudo-random function) but it only support strict "AND" policy of pre-determined authorities. Lin *et al.* [26] proposed a decentralized scheme based on threshold mechanism. In this scheme, the set of authorities is pre-determined and it requires the interaction among the authorities during the system setup. In [10], Lewko *et al.* proposed a new comprehensive scheme, which does not require any central authority. However, they did not consider attribute revocation problem.

There are a number of works about the revocation in ABE systems in the cryptography literature [2]–[6]. However, these methods either only support the user level revocation or rely on the server to conduct the attribute revocation. Moreover, these attribute revocation methods are designed only for ABE systems with single authority. Ruj *et al.* [13] designed a DACC scheme and proposed an attribute revocation method for the Lewko and Waters' decentralized ABE scheme. However, their attribute revocation method incurs a heavy communication



(a) Encryption (b) Encryption (c) Decryption (d) Decryption (e) Re-encryption/Update

Fig. 2. Comparison of Encryption, Decryption and Ciphertext Re-encryption/Update Time

cost since it requires the data owner to transmit a new ciphertext component to every non-revoked user. Li *et al.* [28] proposed an attribute revocation method for multi-authority ABE systems, but their methods is only for KP-ABE systems.

Green *et al.* [11] proposed two ABE schemes that outsource the decryption to the server. In their schemes, the authority separate the traditional secret key into a user secret key and a transformation key. However, their schemes are designed only for the single authority systems and do not support for the multi-authority systems. That is because each authority may generate different user's secret key, such that the transformation keys cannot be combined together to transform the ciphertext into a correct intermediate value.

VI. Conclusion

In this paper, we proposed an effective data access control scheme for multi-authority cloud storage systems, DAC-MACS. We also constructed a new multi-authority CP-ABE scheme, in which the main computation of decryption is outsourced to the server. We further designed an efficient attribute revocation method that can achieve both forward security and backward security. However, in DAC-MACS, all the users' secrets $\{u_{uid}\}$ are revealed to each *AA*. If the revoked user corrupts any *AA* and some non-revoked users, it can derive the key update key easily and apply it to update its secret key. This security weakness will be addressed in our future work.

Acknowledgment

This work is supported by Research Grants Council of Hong Kong [Project No. CityU 114112] and in part by US National Science Foundation under grants CNS-1262277 and CNS-1116939.

References

[1] P. Mell and T. Grance, "The NIST definition of cloud computing," National Institute of Standards and Technology, Tech. Rep., 2009.

[2] J. Bethencourt, A. Sahai, and B. Waters, "Ciphertext-policy attribute-based encryption," in *S&P'07*. IEEE Computer Society, 2007, pp. 321–334.

[3] B. Waters, "Ciphertext-policy attribute-based encryption: An expressive, efficient, and provably secure realization," in *PKC'11*. Springer, 2011, pp. 53–70.

[4] V. Goyal, A. Jain, O. Pandey, and A. Sahai, "Bounded ciphertext policy attribute based encryption," in *ICALP'08*. Springer, 2008, pp. 579–591.

[5] R. Ostrovsky, A. Sahai, and B. Waters, "Attribute-based encryption with non-monotonic access structures," in *CCS'07*. ACM, 2007, pp. 195–203.

[6] A. B. Lewko, T. Okamoto, A. Sahai, K. Takashima, and B. Waters, "Fully secure functional encryption: Attribute-based encryption and (hierarchical) inner product encryption," in *EUROCRYPT'10*. Springer, 2010, pp. 62–91.

[7] M. Chase, "Multi-authority attribute based encryption," in *TCC'07*. Springer, 2007, pp. 515–534.

[8] S. Müller, S. Katzenbeisser, and C. Eckert, "Distributed attribute-based encryption," in *ICISC'08*. Springer, 2008, pp. 20–36.

[9] M. Chase and S. S. M. Chow, "Improving privacy and security in multi-authority attribute-based encryption," in *CCS'09*. ACM, 2009, pp. 121–130.

[10] A. B. Lewko and B. Waters, "Decentralizing attribute-based encryption," in *EUROCRYPT'11*. Springer, 2011, pp. 568–588.

[11] M. Green, S. Hohenberger, and B. Waters, "Outsourcing the decryption of abe ciphertexts," in *Proceedings of the 20th USENIX Security Symposium*. USENIX Association, 2011.

[12] J. Hur and D. K. Noh, "Attribute-based access control with efficient revocation in data outsourcing systems," *IEEE Trans. Parallel Distrib. Syst.*, vol. 22, no. 7, pp. 1214–1221, 2011.

[13] S. Ruj, A. Nayak, and I. Stojmenovic, "DACC: Distributed Access Control in Clouds," in *TrustCom'11*. IEEE, 2011, pp. 91–98.

[14] K. Yang, X. Jia, and K. Ren, "Dac-macs: Effective data access control for multi-authority cloud storage systems," *IACR Cryptology ePrint Archive*, vol. 419, pp. 1–12, 2012.

[15] M. Kallahalla, E. Riedel, R. Swaminathan, Q. Wang, and K. Fu, "Plutus: Scalable secure file sharing on untrusted storage," in *FAST'03*. USENIX, 2003.

[16] E.-J. Goh, H. Shacham, N. Modadugu, and D. Boneh, "Sirius: Securing remote untrusted storage," in *NDSS'03*. The Internet Society, 2003.

[17] D. Naor, M. Naor, and J. Lotspiech, "Revocation and tracing schemes for stateless receivers," *Electronic Colloquium on Computational Complexity (ECCC)*, 2002.

[18] J. Benaloh, M. Chase, E. Horvitz, and K. Lauter, "Patient controlled encryption: ensuring privacy of electronic medical records," in *CCSW'09*. ACM, 2009, pp. 103–114.

[19] C. Dong, G. Russello, and N. Dulay, "Shared and searchable encrypted data for untrusted servers," *Journal of Computer Security*, vol. 19, no. 3, pp. 367–397, 2011.

[20] E. Damiani, S. D. C. di Vimercati, S. Foresti, S. Jajodia, S. Paraboschi, and P. Samarati, "Key management for multi-user encrypted databases," in *StorageSS'05*. ACM, 2005, pp. 74–83.

[21] W. Wang, Z. Li, R. Owens, and B. K. Bhargava, "Secure and efficient access to outsourced data," in *CCSW'09*. ACM, 2009, pp. 55–66.

[22] A. Sahai and B. Waters, "Fuzzy identity-based encryption," in *EUROCRYPT'05*. Springer, 2005, pp. 457–473.

[23] V. Goyal, O. Pandey, A. Sahai, and B. Waters, "Attribute-based encryption for fine-grained access control of encrypted data," in *CCS'06*. ACM, 2006, pp. 89–98.

[24] S. Yu, C. Wang, K. Ren, and W. Lou, "Attribute based data sharing with attribute revocation," in *ASIACCS'10*. ACM, 2010, pp. 261–270.

[25] S. Jahid, P. Mittal, and N. Borisov, "Easier: encryption-based access control in social networks with efficient revocation," in *ASIACCS'11*. ACM, 2011, pp. 411–415.

[26] H. Lin, Z. Cao, X. Liang, and J. Shao, "Secure threshold multi authority attribute based encryption without a central authority," *Inf. Sci.*, vol. 180, no. 13, pp. 2618–2632, 2010.

[27] J. Li, Q. Huang, X. Chen, S. S. M. Chow, D. S. Wong, and D. Xie, "Multi-authority ciphertext-policy attribute-based encryption with accountability," in *ASIACCS'11*. ACM, 2011, pp. 386–390.

[28] M. Li, S. Yu, Y. Zheng, K. Ren, and W. Lou, "Scalable and secure sharing of personal health records in cloud computing using attribute-based encryption," *IEEE Trans. Parallel Distrib. Syst.*, 2012.